Reviewer Pack — Minimal Geometric Entropy and Communication Complexity Growt…

Entry 97be69dd8c4f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 03:35:18 UTC

Minimal Geometric Entropy and Communication Complexity Growth in Hyperbolic Graphs

Entry ID: `97be69dd8c4f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 03:35:18 UTC

1. Conjecture

Field A (mathematical branch): Hyperbolic Geometry **Field B** (complexity object): Communication Complexity

Statement:

> For any hyperbolic graph G with n vertices, the communication complexity of computing a global property (e.g., minimum spanning tree or maximum matching) on G is Θ(n log n / log log n), and this bound is tight.

Rationale (proposer's reasoning):

> Hyperbolic geometry provides a natural setting for studying distributed algorithms in networks. The conjecture posits that communication complexity in hyperbolic graphs exhibits logarithmic dependence on the graph's dimension, which could expose fundamental structural properties of hyperbolic networks and lead to improved algorithms.

Taxonomy category: `HyperbolicGeometry` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `749383349d48dfca`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The communication complexity of computing a global property on a hyperbolic graph G with n vertices is within ±10% of Θ(n log n / log log n), and there are no instances where the communication complexity exceeds this by more than 1000 bits.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"hyperbolic geometry" AND "communication complexity" AND minimal geometric entropy` - `"computing global property" in hyperbolic graphs AND communication complexity` - `"spanning tree" OR "maximum matching" AND communication complexity AND hyperbolic geometry`

Top relevant hits considered: - [s2:a6ae230cef877cad863a11d41c6234830c905733] Algorithms and computation : 16th International Symposium, ISAAC 2005, Sanya, Hainan, China, December 19-21, 2005 : proc - [s2:e5bc32d507cc9074a0edbe512fd183193f8ac605] Euro-Par'96 : parallel processing : Second International Euro-Par Conference, Lyon, France, August 26-29, 1996 : proceed

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_hyperbolic_graph(n):
        # Simplified Gromov-Nagaev construction for hyperbolic graph
        nodes = list(range(n))
        edges = []
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    edges.append((i, j))
        return nodes, edges

    def communication_complexity(nodes, edges):
        # Simplified simulation of communication complexity
        return len(edges) * 2  # Each edge requires 2 bits for sending and receiving

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        nodes, edges = generate_hyperbolic_graph(n)
        cc = communication_complexity(nodes, edges)
        results.append({
```

```
            "n": n,
            "communication_complexity": cc
        })

    mean_cc = sum(result["communication_complexity"] for result in results) / len(results)
    expected_bound = Fraction(n * math.log(n) / math.log(math.log(n)), 1) if n > 1 else 0

    return {
        "metric_name": "Communication Complexity",
        "metric_value": mean_cc,
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": abs(mean_cc - expected_bound) <= Fraction(10 * n, 100) and all(abs(cc
        "counterexample": "" if all(abs(cc - expected_bound) <= 1000 for cc in [result["communicat
    }

if __name__ == "__main__":
    import sys
    seeds = [int(sys.argv[1])] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_cc = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_cc} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Communication complexity exceeds bound by more
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d5658700.py", line 64, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d5658700.py", line 47, in run_trial
    expected_bound = Fraction(n * math.log(n) / math.log(math.log(n)), 1) if n > 1 else 0
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be "
TypeError: both arguments should be Rational instances
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents a definitive evaluation of the conjecture. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15009
2	preregistration	ollama_remote	glm4:latest	0	0	20496
3	novelty	ollama_remote	glm4:latest	0	0	8633
4	novelty	ollama_remote	glm4:latest	0	0	9941
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17123
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8994
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9661
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28004
9	judge	ollama_remote	glm4:latest	0	0	14221

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 132085 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/97be69dd8c4f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/97be69dd8c4f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/97be69dd8c4f.tar.gz` (if generated)